

DEEP STUDY

Docker DKR.01

تاریخچه و مدل ذهنی کانتینر

DS-DKR-01	شناسه سند
1.0.0	نسخه
canonical-study	وضعیت
DevOps / Docker	مسیر
2026-08-11	Research cutoff
Core	سطح

این سند اولین واحد مطالعاتی رسمی مسیر Docker Mastery است. هدف آن ساختن پایه‌ای است که تمام درس‌های بعدی Docker روی آن سوار می‌شوند.

فهرست مطالب

6	راهنمای استفاده از این درس
6	زمان و ریزدانشی این واحد
6	Scope Contract
6	مشخصات سند
7	داخل Scope
7	خارج از Scope
8	بعد از تسلط به این سند هنوز چه چیزی بلد نیستی؟
8	هدف‌های یادگیری
8	۱. مسئله اصلی: چرا اصلاً کانتینر؟
9	۱.۱ بسته‌بندی وابستگی‌های فضای کاربر (userspace)
9	۱.۲ جدا کردن محیط اجرای برنامه‌ها
9	۱.۳ تبدیل محیط اجرا به خروجی قابل تکرار (artifact)
10	۱.۴ اجرای متراکم‌تر workload
10	۱.۵ ساده‌تر شدن تحویل و انتشار نرم‌افزار
10	۲. تاریخچه‌ای که واقعاً به فهم کمک می‌کند
10	۲.۱ قبل از Docker، primitive ها وجود داشتند
11	۲.۲ نقطه مهم Docker: تجربه استفاده و delivery
11	۲.۳ استانداردسازی با OCI
11	۲.۴ Moby و جدا شدن لایه‌های open source
12	۳. واژه «container» دقیقاً یعنی چه؟
12	۳.۱ تعریف عملی برای این مسیر
12	۳.۲ کانتینر object واحد kernel نیست
12	۴. VM و Process، Container، System Container
13	۴.۱ Process چیست؟
13	۴.۲ Application container چیست؟
13	۴.۳ System container چیست؟
13	۴.۴ VM چیست؟
13	۴.۵ جدول مقایسه
14	۵. چرا «Container = Lightweight VM» اشتباه است؟

14	 ۵.۱ VM از ماشین شروع می‌کنی
14	 ۵.۲ در container از process شروع می‌کنی
15	 ۵.۳ kernel مشترک هم مزیت است هم محدودیت
15	۶. اجزای مدل ذهنی کانینر
15	 ۶.۱ Process یا processها
15	 ۶.۲ Namespace: جدا کردن view
16	 ۶.۳ cgroup: سازمان‌دهی و کنترل منابع
16	 ۶.۴ Filesystem view و mountها
16	 ۶.۵ Capabilities و Credentials
16	 ۶.۶ Security policy
16	 ۶.۷ Runtime configuration
17	۷. Isolation صفر و یک نیست
17	۸. Kernel مشترک (Shared Kernel) دقیقاً چه معنایی دارد؟
17	 ۸.۱ چرا داخل container فایل‌های bin/ و lib/ متفاوت می‌بینیم؟
17	 ۸.۲ پس آیا می‌توان Linux container را مستقیم روی Windows اجرا کرد؟
18	 ۸.۳ آیا shared kernel یعنی همه containerها همه چیز را از هم می‌بینند؟
18	۹. قابلیت جابه‌جایی (Portability): واقعیت دقیق‌تر از شعار «Run Anywhere»
18	 ۹.۱ چه چیزی portable تر می‌شود؟
18	 ۹.۲ چه چیزهایی هنوز به platform وابسته‌اند؟
19	 ۹.۳ تعریف بهتر portability
19	۱۰. Image و Container یکی نیستند
20	۱۱. نقشه اکوسیستم: چه چیزی Docker است و چه چیزی نیست؟
20	 ۱۱.۱ Docker به‌عنوان شرکت و product ecosystem
20	 ۱۱.۲ Docker Engine
20	 ۱۱.۳ Moby
20	 ۱۱.۴ OCI
20	 ۱۱.۵ runc
20	 ۱۱.۶ containerd
21	 ۱۱.۷ Docker Compose
21	 ۱۱.۸ Kubernetes
21	 ۱۱.۹ نقشه ساده
22	۱۲. Linux Container در برابر Windows Container
22	 ۱۲.۱ Linux

22	Windows Process Isolation ۱۲.۲
22	Windows Hyper-V Isolation ۱۲.۳
22	۱۳. امنیت (Security): کانتینر را security boundary مطلق فرض نکن
23	۱۴. یک مدل ذهنی نهایی
24	۱۵. Lab اصلی: process و namespace را واقعاً ببین
24	۱۵.۱ پیش‌نیاز
24	۱۵.۲ قبل از اجرا: پیش‌بینی
24	۱۵.۳ یک container ساده بساز
25	۱۵.۴ PID process اصلی را از دید host پیدا کن
25	۱۵.۵ داخل container چه می‌بینی؟
25	۱۵.۶ namespace ID ها را مقایسه کن
26	۱۵.۷ چه چیزی باید مشاهده کنی؟
26	۱۵.۸ kernel را مقایسه کن
26	۱۵.۹ آزمایش اختیاری: یک مرز isolation را عمداً share کن
27	۱۵.۱۰ پاک‌سازی
27	۱۵.۱۱ گزارش lab
27	۱۶. پنج مدل ذهنی اشتباه
27	۱۶.۱ «Container همان VM سبک است»
27	۱۶.۲ «Container یعنی namespace»
28	۱۶.۳ «Container کاملاً self-contained است»
28	۱۶.۴ «اگر داخل container root باشیم یعنی root host هستیم»
28	۱۶.۵ «Docker و container یک چیزند»
28	۱۷. سوال‌های پیش‌بینی و چالش
28	چالش ۱
28	چالش ۲
28	چالش ۳
29	چالش ۴
29	چالش ۵
29	چالش ۶
29	۱۸. پاسخ مصاحبه در سه عمق
29	۱۸.۱ سوال: Container چیست؟ - پاسخ ۳۰ ثانیه
29	۱۸.۲ پاسخ حدود ۳ دقیقه
30	۱۸.۳ پاسخ ۳۰ دقیقه‌ای: نقشه بحث

30	۱۹. تمرین دونفره: دفاع و نقد (Attack / Defend)
30	۲۰. سوال‌های بازگویی (Teach-back)
31	۲۱. چک‌لیست تسلط
31	فهم مفهومی
31	نقشه اکوسیستم
32	Lab
32	ارتباط و مصاحبه
33	۲۲. تمرین خواندن منبع و Spec
33	۲۳. سوال‌های باز که فعلاً باید نگه داری
33	۲۴. جمع‌بندی یک صفحه‌ای
34	۲۵. قدم بعدی
34	منابع
37	Changelog

راهنمای استفاده از این درس

این PDF اولین درس رسمی مسیر **Docker Mastery** است. قرار نیست فقط چند تعریف حفظ کنی. وقتی این درس تمام شد باید بتوانی بدون تکیه به متن، توضیح بدهی «کانتینر چیست، چه مشکلی را حل می‌کند، چه چیزی نیست، و Docker در کل این داستان کجا قرار می‌گیرد».

برای این درس سه مرحله پیشنهاد می‌شود:

1. یک بار متن را با تمرکز روی مدل ذهنی بخوان. در این مرحله لازم نیست همه جزئیات را حفظ کنی.
2. آزمایش عملی را انجام بده و قبل از هر دستور، نتیجه را پیش‌بینی کن.
3. PDF را ببند و موضوع را برای شریک مطالعات توضیح بده. هر جا نتوانستی مرز یک مفهوم را روشن کنی، همان بخش را دوباره بخوان.

معیار پایان درس: اگر فقط بتوانی بگویی «کانتینر سبک‌تر از VM است»، این درس تمام نشده است. باید بتوانی تفاوت process، application container، system container، و VM را توضیح بدهی و نشان بدهی که در Linux چیزی به نام یک primitive واحد به اسم container وجود ندارد؛ کانتینر از کنار هم قرار گرفتن چند سازوکار سیستم‌عامل ساخته می‌شود.

زمان و ریزدانی این واحد

این درس عمداً یک واحد مستقل و متوسط است. هدف این است که تمام‌کردن آن یک پیشرفت واقعی باشد، نه فقط خواندن یک تکه کوتاه از یک موضوع بزرگ. برای بیشتر افراد، یک مطالعه جدی همراه با lab و teach-back حدود ۳ تا ۶ ساعت زمان فعال می‌خواهد. اگر بخش‌های Linux برایت جدید هستند، طبیعی است که بیشتر طول بکشد. این عدد deadline نیست. معیار اصلی، **شواهد تسلط** است که در انتهای سند آمده است.

Scope Contract

مشخصات سند

مورد	مقدار
Document ID	DS-DKR-01
Syllabus owner	DKR.01
عنوان	تاریخچه و مدل ذهنی کانتینر

مورد	مقدار
سطح	Core
نسخه	1.0.0
Research cutoff	2026-08-11
پیش‌نیاز مفهومی	META.DKR.01؛ آشنایی عمومی با process و سیستم‌عامل مفید است ولی اجباری نیست
پیش‌نیاز lab	یک Linux host یا Linux VM که Docker روی آن قابل اجرا باشد
سند بعدی	DKR.02 - نصب، Platforms و Distribution

داخل Scope

این سند این موضوع‌ها را کامل در سطح مدل ذهنی پایه پوشش می‌دهد:

- مسئله‌ای که کانتینرها برای توسعه و اجرای نرم‌افزار حل می‌کنند؛
- تفاوت application container، system container، process و VM؛
- دلیل اینکه «container = lightweight VM» مدل دقیقی نیست؛
- کانتینر به‌عنوان ترکیب credentials، filesystem view، cgroup، namespace، process و policyهای امنیتی؛
- پیامد shared-kernel بودن؛
- معنی واقعی portability و محدودیت‌های آن؛
- جایگاه Compose، runc، containerd، OCI، Moby، Docker Engine و Kubernetes؛
- تفاوت کلی Linux container و Windows container؛
- یک آزمایش واقعی برای دیدن process و namespaceهای یک کانتینر از host.

خارج از Scope

این موارد فقط معرفی می‌شوند و owner اصلی آن‌ها سند‌های بعدی هستند:

- نصب و انتخاب Docker Engine/Desktop: DKR.02
- مسیر دقیق docker CLI -> API -> dockerd -> containerd -> runc: DKR.03
- lifecycle کامل کانتینر: DKR.05
- TTY، signal، PID 1 و I/O: DKR.06
- image layer، manifest و digest: DKR.09
- networking: DKR.23 تا DKR.27
- Linux namespaces: LNX.02
- cgroup v2: LNX.04

• mount/rootfs/OverlayFS : LNX.05 و LNX.06

• LSM و capabilities، seccomp : LNX.09

• Runtime Internals : runc و containerd فاز

مرز مهم: این درس قرار نیست همه چیز درباره namespace و cgroup را آموزش دهد. اگر همین جا وارد جزئیات کامل آن ها شویم، هم ریزدانی پروژه خراب می شود و هم درس های owner بعدی تکراری می شوند. اینجا فقط باید نقش هر سازوکار را آن قدر بفهمی که مدل «کانتینر» اشتباه شکل نگیرد.

بعد از تسلط به این سند هنوز چه چیزی بلد نیستی؟

بعد از این درس هنوز Docker operator نیستی. هنوز نصب، storage، networking، image، CLI و production debugging را نخوانده ای. چیزی که به دست می آوری **فونداسیون فکری** است: از این به بعد هر feature را می توانی روی یک مدل درست سوار کنی.

هدف های یادگیری

وقتی این درس تمام شد باید بتوانی:

1. توضیح بدهی چرا containerization فقط «سبک تر کردن VM» نیست و چه مسئله های مهندسی نرم افزار را هدف می گیرد.
2. process، application container، system container و VM را بدون تعریف های مبهم از هم جدا کنی.
3. توضیح بدهی که یک Linux container از چند primitive سیستم عامل ساخته می شود و خودش primitive واحد kernel نیست.
4. shared kernel را هم به عنوان مزیت و هم محدودیت تحلیل کنی.
5. portability کانتینر را دقیق و بدون شعار «هرجا اجرا می شود» توضیح بدهی.
6. مرز Compose، runc، containerd، OCI، Moby، Docker و Kubernetes را در یک نقشه ساده نشان بدهی.
7. با docker inspect، /proc و lsns یک process کانتینری را از host مشاهده کنی و نتیجه را توضیح بدهی.

۱. مسئله اصلی: چرا اصلاً کانتینر؟

برای فهم یک ابزار، بهتر است از command شروع نکنیم. از مشکل شروع کنیم.

فرض کن یک backend ساده داری. روی لپ تاپ تو با Python 3.x، یک نسخه خاص OpenSSL، چند library سیستم عامل و یک configuration مشخص کار می کند. حالا این برنامه باید در CI، لپ تاپ نفر دوم تیم، staging و production اجرا شود.

اگر هر محیط را جداگانه و دستی بسازیم، چند نوع drift به وجود می‌آید:

- نسخه runtime فرق می‌کند؛
- libraryهای سیستم فرق می‌کنند؛
- configurationها ناخواسته تغییر می‌کنند؛
- dependencyهای یک پروژه با پروژه دیگر conflict پیدا می‌کنند؛
- deployment به مجموعه‌ای از دستورهای دستی تبدیل می‌شود؛
- بازتولید یک محیط خراب سخت می‌شود.

Docker در مستندات خودش container را یک محیط نسبتاً ایزوله برای package و run کردن application معرفی می‌کند و روی توسعه، انتقال و اجرای سازگارتر workload تأکید دارد. [R1][R2]

اما این را دقیق‌تر کنیم. کانتینرها معمولاً پنج نیاز را کنار هم پاسخ می‌دهند.

۱.۱ بسته‌بندی وابستگی‌های فضای کاربر (userspace)

به جای اینکه بگوییم:

«اول این نسخه Python را نصب کن، بعد این packageها، بعد این فایل config را در فلان مسیر بگذار...»

می‌توانی userspace لازم برنامه را داخل یک image توصیف و توزیع کنی. این userspace شامل فایل‌های application، binary، libraryها و configurationهای لازم می‌شود. Docker image را یک package استاندارد شامل فایل‌ها، binary، libraryها و configurationهای موردنیاز برای اجرای container توصیف می‌کند. [R3]

این به معنی بسته‌بندی kernel نیست. این نکته بعداً بسیار مهم می‌شود.

۱.۲ جدا کردن محیط اجرای برنامه‌ها

دو برنامه ممکن است نسخه‌های متفاوت library یا تنظیمات متفاوت بخواهند. اگر همه چیز مستقیم روی یک host نصب شود، محیط‌ها به هم نزدیک می‌شوند و احتمال conflict بالا می‌رود.

کانتینر برای process یک view جدا از بعضی منابع سیستم می‌سازد. مثلاً process می‌تواند PID space، hostname، mountها یا network stack متفاوتی ببیند. در Linux، namespaceها دقیقاً برای ساخت viewهای جدا از منابع سراسری سیستم استفاده می‌شوند و یکی از کاربردهای اصلی آن‌ها پیاده‌سازی container است. [R4]

۱.۳ تبدیل محیط اجرا به خروجی قابل تکرار (artifact)

اگر image و configuration مشخص باشند، ساخت یک container جدید معمولاً از ساخت دستی همان محیط قابل تکرارتر است. Kubernetes نیز container را به‌عنوان روشی برای package کردن application همراه runtime dependencyهایش توضیح می‌دهد و روی repeatability تأکید می‌کند. [R5]

نکته مهم این است که **repeatable** با **identical under every host condition** فرق دارد. Host kernel, architecture, device, volume, network policy هنوز مهم‌اند.

۱.۴ اجرای متراکم‌تر workloadها

در مدل سنتی VM، برای هر guest معمولاً یک kernel و مجموعه‌ای از سرویس‌های سیستم‌عامل نیز اجرا می‌شود. در containerهای shared-kernel، چند workload می‌توانند از kernel host استفاده کنند. به همین دلیل overhead معمولاً کمتر است و می‌توان workloadهای بیشتری را روی یک host قرار داد. Docker همین موضوع را یکی از مزیت‌های اصلی container در مقایسه با VM می‌داند. [R1][R2]

اما «کم‌مصرف‌تر» به معنی «بدون هزینه» نیست. container همچنان CPU، memory، I/O، network و storage مصرف می‌کند و ممکن است overheadهای filesystem یا networking داشته باشد.

۱.۵ ساده‌تر شدن تحویل و انتشار نرم‌افزار

Container image می‌تواند واحدی برای pull، push، scan، test، build و deploy باشد. این باعث می‌شود مرز بین توسعه و عملیات واضح‌تر شود. به جای تحویل یک لیست مبهم از dependencyها، artifact مشخص‌تری تحویل می‌دهی.

مدل ساده: containerization تلاش می‌کند «برنامه + userspace لازم + runtime configuration» را از وضعیت تصادفی host جدا کند و اجرای آن را قابل تکرارتر، ایزوله‌تر و قابل انتقال‌تر کند.

۲. تاریخچه‌ای که واقعاً به فهم کمک می‌کند

هدف این بخش حفظ سال‌ها نیست. فقط باید بفهمی Docker از خلأ به وجود نیامد.

۲.۱ قبل از Docker، primitiveها وجود داشتند

Linux سال‌ها قبل از محبوب‌شدن Docker سازوکارهایی برای isolation و resource control داشت. namespaceها view منابع را جدا می‌کنند و cgroupها processها را در hierarchy سازمان می‌دهند و مصرف منابع را کنترل می‌کنند. [R4][R6]

ابزارهای دیگری مثل LXC نیز از همین primitiveها برای ساخت محیط‌های containerized استفاده می‌کردند. پروژه Linux Containers امروز هم system container را محیطی شبیه یک Linux کامل می‌داند که kernel host را share می‌کند. [R7]

پس Docker «ایده isolation processها» را اختراع نکرد.

۲.۲ نقطه مهم Docker: تجربه استفاده و delivery

Docker در سال ۲۰۱۳ به صورت open source معرفی شد. خود Docker توضیح می‌دهد که فناوری‌اش روی concept‌های قبلی، مخصوصاً cgroups و namespaces، بنا شد ولی تمرکز را روی نیاز developer و operator برای جدا کردن dependency برنامه از infrastructure گذاشت. [R8]

این تغییر مهم بود: container فقط یک طرفند kernel نبود؛ تبدیل شد به workflow روزمره build، ship و run.

به زبان ساده، Docker سه چیز را کنار هم آورد:

- یک تجربه کاربری قابل استفاده برای developer؛
- image و distribution workflow؛
- یک engine برای ساخت، انتقال و اجرای containerها.

۲.۳ استانداردسازی با OCI

با بزرگ‌شدن اکوسیستم، اگر هر vendor format و runtime کاملاً مخصوص خودش را می‌ساخت، interoperability سخت می‌شد.

Open Container Initiative یا OCI در ژوئن ۲۰۱۵ با مشارکت Docker و دیگر شرکت‌ها شکل گرفت تا استانداردهای باز برای runtime، image و distribution ایجاد کند. OCI امروز سه specification اصلی دارد: Runtime Specification، Image Specification و [R9] Distribution Specification.

این یعنی مفهوم container دیگر به یک product خاص محدود نیست.

۲.۴ Moby و جدا شدن لایه‌های open source

در ۲۰۱۷، Docker پروژه Moby را به عنوان فضای open source برای componentها و ساخت سیستم‌های container-based معرفی کرد. Moby امروز خودش را مجموعه‌ای modular از building blockها می‌داند و Docker از آن به عنوان upstream برای product Docker استفاده می‌کند. [R10]

هم‌زمان componentهایی مثل containerd و runc نقش‌های مشخص‌تری گرفتند. این تفکیک برای ما مهم است چون هدف Deep Study رسیدن از UX بالا تا implementation پایین است.

اگر از تاریخ فقط یک نکته نگه می‌داری، این باشد: Docker کانتینر را از یک مجموعه primitive سیستم‌عامل به یک workflow استاندارد و قابل استفاده برای توسعه و delivery نزدیک کرد؛ بعد OCI و پروژه‌های runtime مرزهای استاندارد و modular را قوی‌تر کردند.

۳. واژه «container» دقیقاً یعنی چه؟

واژه container overloaded است. حتی Kubernetes هم هشدار می‌دهد که این واژه ممکن است در context‌های مختلف معنی متفاوتی داشته باشد. [R5]

گاهی منظور یک process در حال اجراست. گاهی یک محیط system-like است. گاهی مردم اشتباهی image را container می‌نامند. بنابراین از همین ابتدا باید context را مشخص کنیم.

۳.۱ تعریف عملی برای این مسیر

در مسیر Docker، وقتی می‌گوییم **کانتینر (container)** معمولاً منظورمان این است:

یک یا چند process که با configuration مشخص در یک محیط محدود و تا حدی ایزوله اجرا می‌شوند و view آن‌ها از process، filesystem، network و دیگر منابع می‌تواند با host یا containerهای دیگر فرق داشته باشد.

این تعریف عمداً نمی‌گوید «یک ماشین کوچک».

۳.۲ کانتینر object واحد kernel نیست

در Linux یک primitive واحد با معنای کامل «container» نداریم. namespace یک primitive است. cgroup یک subsystem است. mount یک سازوکار جداست. credentials و capability سازوکارهای دیگرند. runtime این‌ها را کنار هم می‌گذارد تا محیطی بسازد که ما به آن container می‌گوییم. [R4][R6][R11]

پس بهتر است کانتینر را یک **composition** ببینی، نه یک object جادویی.

فرمول ذهنی این درس:

Container $\sim$ Process(es) + isolated views + resource policy + filesystem view + credentials + security policy + runtime configuration

علامت $\sim$ عمدی است. این یک تعریف formal نیست؛ یک مدل ذهنی است.

۴. VM، Process، Container، System Container

این بخش یکی از مهم‌ترین بخش‌های کل مسیر است.

۴.۱ Process چیست؟

یک process اجرای زنده یک program است. process state دارد، PID دارد، memory space دارد، file descriptor دارد و با credentialهای مشخص اجرا می‌شود.

اما یک process معمولی روی host معمولاً بسیاری از viewهای global سیستم را با processهای دیگر share می‌کند: PID namespace، mount namespace، network namespace و غیره. کانتینر از همین process شروع می‌شود، نه از یک virtual computer.

۴.۲ Application container چیست؟

Application container مدلی است که Docker آن را popular کرد: معمولاً یک application یا component مشخص را package و اجرا می‌کند. هدف اصلی، اجرای یک workload است، نه شبیه‌سازی یک ماشین کامل. این به معنی «فقط یک process همیشه» نیست. یک container می‌تواند چند process داشته باشد. قانون «one process per container» بیشتر یک guideline طراحی است تا قانون kernel.

۴.۳ System container چیست؟

System container تلاش می‌کند یک userspace کامل‌تر شبیه یک Linux system ارائه دهد؛ ممکن است init system و چند سرویس داخلش اجرا شوند. Incus تفاوت application container و system container را همین‌طور توضیح می‌دهد: application container معمولاً یک app/component را package می‌کند، ولی system container تجربه نزدیک‌تری به یک سیستم Linux کامل می‌دهد و همچنان kernel host را share می‌کند. [R7] این distinction مهم است چون وقتی می‌شنوی «container»، همیشه نباید فرض کنی دقیقاً مدل Docker application container منظور است.

۴.۴ VM چیست؟

Virtual Machine معمولاً یک guest OS کامل با kernel خودش دارد و روی virtual hardware اجرا می‌شود. Hypervisor مرز بین VMها و host را ایجاد می‌کند. در مقابل، application container معمولاً از kernel host استفاده می‌کند. Docker و Microsoft هر دو در مقایسه container و VM روی همین تفاوت kernel مشترک در برابر guest kernel مستقل تأکید می‌کنند. [R1][R12]

۴.۵ جدول مقایسه

ویژگی	Process معمولی	Application Container	System Container	VM
kernel مستقل	خیر	معمولاً خیر	خیر	بله
isolation قابل تنظیم	کم	بله	بله	قوی‌تر و در سطح hypervisor

ویژگی	Process معمولی	Application Container	System Container	VM
userspace مستقل	معمولاً خیر	بله، متناسب با app	بله، نزدیک Linux کامل	بله، OS کامل
چند سرویس	ممکن	ممکن ولی معمولاً محدود	معمولاً بله	بله
startup معمول	خیلی سریع	سریع	سریع	معمولاً سنگین تر
footprint پایه	کم	کم	کم تا متوسط	بیشتر
use case اصلی	اجرای برنامه روی host	package/run component	محیط system-like	ماشین/OS مستقل

این جدول یک simplification است. performance و security واقعی به workload و configuration بستگی دارد.

۵. چرا «Container = Lightweight VM» اشتباه است؟

این جمله در شروع شاید برای intuition بد نباشد، ولی اگر در ذهن بماند بعداً دهها سوءبرداشت می‌سازد.

۵.۱ در VM از ماشین شروع می‌کنی

در VM ابتدا یک machine abstraction داری: CPU مجازی، device، memoryهای مجازی و guest kernel. سپس processهای application داخل guest OS اجرا می‌شوند.

۵.۲ در container از process شروع می‌کنی

در container معمولاً از process شروع می‌کنی و view و دسترسی آن را محدود می‌کنی. یعنی جهت ذهنی متفاوت است:

VM:
virtual hardware
-> guest kernel
-> guest userspace
-> application process

Container:
host kernel
-> isolated/configured process environment
-> application process(es)

این تفاوت روی compatibility، performance، security، debugging و networking اثر می‌گذارد.

۵.۳ kernel مشترک هم مزیت است هم محدودیت

مزیت‌ها: startup سریع‌تر، footprint کمتر برای لایه سیستم‌عامل، تراکم اجرای بهتر و ارتباط مستقیم‌تر با kernel host. محدودیت‌ها: نمی‌توانی هر kernel دلخواهی را داخل یک process-isolated container اجرا کنی؛ آسیب‌پذیری یا featureهای kernel host می‌توانند روی containerها اثر بگذارند؛ و بعضی workloadها که kernel module یا قابلیت خاص می‌خواهند اصلاً انتخاب خوبی برای container نیستند. پس kernel مشترک فقط یک جمله برای مصاحبه نیست؛ یکی از مهم‌ترین trade-offهای معماری container است.

۶. اجزای مدل ذهنی کانتینر

حالا فرمول اصلی را باز می‌کنیم.

۶.۱ Process یا processها

در نهایت چیزی باید CPU بگیرد و code اجرا کند. این همان process است. اگر process اصلی تمام شود، در مدل رایج container، application container هم به state متوقف‌شده می‌رود. جزئیات آن در DKR.05 و DKR.06 می‌آید.

۶.۲ Namespace: جدا کردن view

Linux namespace یک global resource را طوری wrap می‌کند که processهای داخل namespace بتوانند instance جداگانه‌ای از آن resource ببینند. Linux manual صریحاً container را یکی از use caseهای namespace معرفی می‌کند. [R4]

مثال‌های مهم:

- PID namespace: view process IDها؛
- mount namespace: mount table؛
- network namespace: interface، route، socket و بخش‌های دیگر network stack؛
- UTS namespace: hostname؛
- IPC namespace: بعضی IPC resourceها؛
- user namespace: mapping هویت user/group؛
- cgroup namespace و time namespace: viewهای مرتبط دیگر.

در این درس کافی است بدانی namespace بیشتر درباره «چه چیزی می‌بینم؟» است.

۶.۳ cgroup: سازمان‌دهی و کنترل منابع

cgroup v2 process hierarchy سازمان می‌دهد و resourceهایی مثل CPU، memory، I/O و تعداد processها را کنترل می‌کند. Kernel documentation همین را نقش اصلی cgroup می‌داند. [R6]

در مدل ساده:

- namespace می‌پرسد: «چه viewی داری؟»

- cgroup می‌پرسد: «چقدر resource می‌توانی مصرف کنی و چطور حساب می‌شود؟»

این دو یکی نیستند.

۶.۴ Filesystem view و mountها

process کانتینری معمولاً root filesystem مخصوص خودش را می‌بیند. Runtime با mountها و filesystem layer این view را می‌سازد. این rootfs لزوماً یک disk کامل یا filesystem مستقل فیزیکی نیست.

جزئیات mount namespace، rootfs، pivot_root و OverlayFS بعداً می‌آیند.

۶.۵ Capabilities و Credentials

اینکه process چه UID/GID و چه privilegeهایی دارد بخش دیگری از isolation است. در Linux، capability privilegeهای سنتی root را به بخش‌های کوچک‌تر تقسیم می‌کنند. در کانتینر معمولاً همه capabilityها به process داده نمی‌شود.

جزئیات این بخش owner سند LNX.09 است.

۶.۶ Security policy

Seccomp می‌تواند syscalls را filter کند. AppArmor یا SELinux می‌توانند policyهای دسترسی دیگری اعمال کنند. Device access نیز قابل محدود کردن است.

پس security کانتینر فقط namespace نیست.

۶.۷ Runtime configuration

همه این primitiveها باید با configuration مشخص کنار هم قرار بگیرند. OCI Runtime Specification یک format استاندارد برای توصیف runtime configuration و lifecycle فراهم می‌کند. در بخش Linux configuration همین spec، namespaceها و cgroupها به عنوان بخش‌های قابل تنظیم runtime دیده می‌شوند. [R11]

نکته‌ای که باید از حفظ بتوانی بگویی: isolation کانتینر حاصل یک لایه نیست. namespace، cgroup، mount credential و policy‌های امنیتی نقش‌های جدا دارند. حذف یا share کردن یکی از این لایه‌ها می‌تواند isolation را تغییر دهد.

۷. Isolation صفر و یک نیست

یک اشتباه رایج این است که بگوییم «process داخل container ایزوله است» و تمام.

Isolation در عمل مجموعه‌ای از انتخاب‌هاست.

یک container می‌تواند network namespace جدا داشته باشد، ولی PID namespace host را share کند. می‌تواند filesystem read-only داشته باشد ولی device خاصی از host را ببیند. می‌تواند root user داخل namespace باشد ولی روی host همان privilege را نداشته باشد.

Docker option‌هایی دارد که بعضی مرزها را عمداً باز می‌کنند؛ مثلاً host networking یا PID namespace host. این یعنی «container بودن» الزاماً یک level ثابت isolation را تضمین نمی‌کند.

این موضوع برای security بسیار مهم است: اسم container به‌تنهایی security posture را تعیین نمی‌کند؛ configuration و host boundary تعیین‌کننده‌اند.

۸. Kernel مشترک (Shared Kernel) دقیقاً چه معنایی دارد؟

وقتی چند Linux container روی یک Linux host process-isolated اجرا می‌شوند، kernel اجرا شده همان kernel host است. داخل image معمولاً userspace وجود دارد، نه kernel جدا.

۸.۱ چرا داخل container فایل‌های /bin و /lib متفاوت می‌بینیم؟

چون userspace و filesystem view متفاوت است. ممکن است Ubuntu host باشد و image از Debian یا Alpine userspace استفاده کند. اما system call در نهایت به kernel host می‌رسند.

۸.۲ پس آیا می‌توان Linux container را مستقیم روی Windows اجرا کرد؟

نه در همان معنای process isolation روی Windows kernel. برای اجرای Linux container روی Windows/macOS معمولاً یک Linux environment یا VM در زیرساخت Docker Desktop وجود دارد. در مقابل، Windows container‌های process-isolated از Windows kernel host استفاده می‌کنند. Microsoft همچنین برای Windows حالت Hyper-V isolation دارد که هر container را داخل یک VM بهینه‌شده با kernel خودش اجرا می‌کند. [R13]

۸.۳ آیا shared kernel یعنی همه containerها همه چیز را از هم می‌بینند؟

نه. namespace ها و policy ها view و دسترسی را جدا می‌کنند. shared kernel یعنی implementation kernel مشترک است، نه اینکه process ها بدون محدودیت همه state همدیگر را ببینند.

۹. قابلیت جابه‌جایی (Portability): واقعیت دقیق‌تر از شعار «Run Anywhere»

Portability یکی از ارزش‌های اصلی container است، اما باید سطح آن را دقیق بفهمی.

۹.۱ چه چیزی portable می‌شود؟

وقتی application و dependency userspace هایش داخل image تعریف می‌شوند، dependency روی package های نصب‌شده host کمتر می‌شود. این باعث می‌شود همان artifact را بتوانی در محیط‌های مختلفی که runtime و platform مناسب دارند اجرا کنی.

۹.۲ چه چیزهایی هنوز به platform وابسته‌اند؟

معماری CPU

یک binary ساخته‌شده برای amd64 لزوماً روی arm64 اجرا نمی‌شود مگر image چندمعماری یا emulation داشته باشی.

نوع OS و kernel

Linux userspace انتظار Linux syscall ABI دارد. Windows container و Linux container یک چیز نیستند.

kernel های feature

یک workload ممکن است namespace، cgroup controller، syscall، یا filesystem feature خاصی بخواهد که روی host دیگر موجود یا enabled نباشد.

device

USB، accelerator، GPU یا device های خاص host-dependent هستند.

storage و network خارجی

routing، DNS، firewall policy، volume driver، Bind mount، و storage backend می‌توانند در محیط‌ها فرق کنند.

rootless configuration و SELinux، AppArmor، seccomp profile، user namespace می‌توانند رفتار برنامه را تغییر دهند.

۹.۳ تعریف بهتر portability

به جای این جمله:

«کانتینر هر جا اجرا می‌شود.»

این را بگو:

«کانتینر dependency های userspace و runtime configuration را استانداردتر می‌کند و در platform های سازگار قابلیت جابه‌جایی را زیاد می‌کند؛ اما kernel، architecture، device policy و host های هنوز بخشی از قرارداد اجرا هستند.»

این جواب هم دقیق‌تر است و هم در مصاحبه نشان می‌دهد تفاوت slogan و engineering را می‌فهمی.

۱۰. Image و Container یکی نیستند

این درس درباره image internals نیست، ولی یک مرز باید همین حالا روشن شود.

Docker می‌گوید container یک instance قابل اجرا از image است. Image package استاندارد از فایل‌ها، binary ها، library ها و configuration است. [R2][R3]

مدل ساده:

Image = artifact/template-ish package
Container = runtime instance created from image + runtime configuration

اما حتی عبارت «template» هم کامل نیست و در DKR.09 اصلاح دقیق‌تری می‌گیری.

Image = zip file نیست. ممکن است export یا archive‌هایی ببینی، ولی image model شامل metadata، configuration و در استاندارد OCI ساختار descriptor/manifest/layer است. فعلاً فقط این مرز را نگه دار؛ internals در DKR.09 می‌آید.

۱۱. نقشه اکوسیستم: چه چیزی Docker است و چه چیزی نیست؟

یکی از علت‌های سردرگمی Docker این است که اسم چند لایه در گفتگوها با هم مخلوط می‌شود.

۱۱.۱ Docker به عنوان شرکت و product ecosystem

Docker نام شرکت و مجموعه product‌هایی مثل Docker Hub، Docker Desktop و ابزارهای توسعه است.

۱۱.۲ Docker Engine

Docker Engine فناوری متن‌باز containerization است. مستندات رسمی آن را یک client-server application معرفی می‌کنند که daemon به نام `dockerd`، API و CLI دارد و object‌هایی مثل `network`، `container`، `image` و `volume` را مدیریت می‌کند. [R14]

جزئیات معماری `owner DKR.03` است.

۱۱.۳ Moby

Moby پروژه open source و modular برای ساخت سیستم‌های container-based است. README پروژه می‌گوید Docker از Moby به عنوان upstream برای Docker product استفاده می‌کند. [R10]

برای کسی که هدف contributor شدن دارد، Moby بسیار مهم است چون بخش بزرگی از implementation Engine را آنجا می‌خوانی.

۱۱.۴ OCI

OCI محصول Docker نیست. یک governance مستقل زیر Linux Foundation برای استاندارد container format و runtime است. سه اصلی آن `Image`، `Runtime` و `Distribution` هستند. [R9]

OCI به ما زبان مشترک می‌دهد؛ مثلاً runtime‌هایی مثل `runc` می‌توانند بر اساس Runtime Spec کار کنند.

۱۱.۵ runc

`runc` یک low-level CLI tool برای spawn و run کردن containerهای Linux طبق OCI specification است. خود پروژه تأکید می‌کند که ابزار سطح پایین است و معمولاً توسط نرم‌افزارهای سطح بالاتر استفاده می‌شود، نه end user. [R15]

۱۱.۶ containerd

`containerd` یک container runtime daemon است که `execution`، `image transfer/storage`، `lifecycle container` و `supervision` را مدیریت می‌کند و برای embedded شدن در سیستم‌های بزرگ‌تر طراحی شده است. [R16]

این یعنی containerd و runc هم سطح نیستند. در درس‌های runtime دقیق می‌بینی containerd بالاتر از low-level OCI runtime می‌ایستد.

Docker Compose ۱۱.۷

Compose ابزار و مدل application برای تعریف چند service و منابع مشترک آن‌هاست. Compose runtime سطح پایین نیست؛ در نهایت با Engine کار می‌کند. جزئیات در DKR.28 و DKR.29.

Kubernetes ۱۱.۸

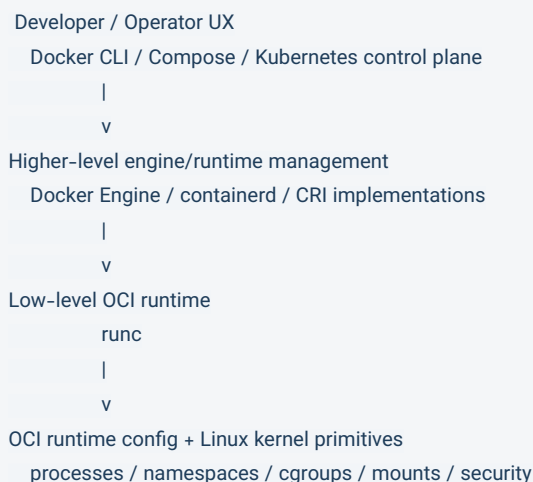
Kubernetes orchestrator است؛ desired state، scheduler و مدیریت workload در cluster را انجام می‌دهد. Kubernetes برای اجرای container روی node به container runtime نیاز دارد و در نسخه‌های فعلی runtime باید با Container Runtime Interface یا CRI سازگار باشد. Kubernetes از integration 1.24 قدیمی dockershim را حذف کرده است. [R17]

پس جمله «Kubernetes جای Docker را گرفت» بیش از حد مبهم است.

- Kubernetes جای Dockerfile یا مفهوم image را نگرفت.
- Kubernetes خودش low-level OCI runtime نیست.
- Kubernetes می‌تواند از containerd یا CRI-O و runtime‌های سازگار استفاده کند.

۱۱.۹ نقشه ساده

این نقشه عمده implementation همه path‌ها را دقیق نشان نمی‌دهد. Docker Engine 29 حتی topology‌های version-sensitive مثل embedded-containerd آزمایشی دارد. مسیر کامل در DKR.03 بررسی می‌شود. Release notes رسمی Docker Engine 29.7.2 آخرین release بررسی‌شده این سند است. [R18]



۱۲. Linux Container در برابر Windows Container

اگر فقط Linux خوانده باشی ممکن است فکر کنی container یعنی namespace های Linux. مفهوم بالاتر container عمومی تر است.

۱۲.۱ Linux

در Linux، container معمولاً با primitive هایی مثل mount، cgroup، namespace و security mechanism های Linux ساخته می شود. `runc` نیز low-level runtime مخصوص Linux است. [R15]

۱۲.۲ Windows Process Isolation

Windows container در حالت process isolation نیز kernel host را share می کند، اما primitive ها و implementation آن Windows-specific هستند. Microsoft می گوید process isolation با namespace، resource control و دیگر isolation technology های Windows ساخته می شود. [R13]

۱۲.۳ Windows Hyper-V Isolation

در Hyper-V isolation، هر Windows container داخل VM بسیار سبک اجرا می شود و عملاً kernel مخصوص خود را دارد. [R13]

این مثال نشان می دهد که «container» یک UX و packaging/runtime model است و implementation isolation می تواند بسته به platform فرق کند.

۱۳. امنیت (Security): کانتینر را security boundary مطلق فرض

نکن

این درس security deep dive نیست، ولی یک تصور غلط باید زود حذف شود.

Shared-kernel container و VM security boundary یکسان نیستند. Microsoft صریحاً process-isolated Windows container و Linux container shared-kernel را برای hostile multi-tenant workload هم سطح یک hypervisor security boundary نمی داند و برای isolation قوی تر hypervisor isolation را توصیه می کند. [R19]

این به معنی «container ناامن است» نیست. به معنی این است که باید threat model داشته باشی.

Security container به چیزهای مختلف وابسته است:

- kernel patch level؛
- privilege های process؛

- capabilityها؛
- seccomp/LSM policy؛
- deviceها و mount؛
- user namespace/rootless؛
- daemon access؛
- image supply chain؛
- network boundary.

پس جمله درست‌تر:

container isolation یک security mechanism مهم است، اما قدرت boundary آن به platform و configuration بستگی دارد و نباید خودکار معادل VM/hypervisor isolation فرض شود.

۱۴. یک مدل ذهنی نهایی

تا اینجا مدل ما باید این شکل را داشته باشد:

```

Application code
|
v
Process(es)
|
+-- filesystem view / mounts
+-- namespaces -> isolated views
+-- cgroups -> resource organization/control
+-- credentials/capabilities
+-- seccomp/LSM/device policy
|
v
Host kernel
|
v
Hardware
  
```

و Docker کجاست؟

Docker لایه‌ای است که ساخت، lifecycle، configuration، package و interaction با این محیط را برای user و developer قابل استفاده می‌کند. خود «Docker» kernel نیست و containerهای kernel را جایگزین نمی‌کند.

اگر interviewer پرسید «**Docker container** واقعاً چیست؟» از اینجا شروع کن: «از دید کاربر یک runtime instance از image است؛ از دید Linux یک process یا مجموعه process است که runtime با namespaceها، mountها، credentialها و policyهای امنیتی برایش محیطی محدود و ایزوله ساخته است.» بعد بسته به زمان، جزئیات را باز کن.

۱۵. Lab اصلی: process و namespace را واقعاً ببین

هدف lab این نیست که چند command حفظ کنی. هدف این است که با چشم ببینی process کانتینری همچنان روی host وجود دارد، ولی view بعضی namespaceها فرق دارد.

۱۵.۱ پیش‌نیاز

این lab را روی Linux host یا Linux VM انجام بده. Docker باید آماده و daemon در حال اجرا باشد. اگر هنوز Docker نصب نکرده‌ای، lab را بعد از DKR.02 برگرد و انجام بده؛ ولی بخش prediction را همین حالا انجام بده.

ابزارهای host:

• docker

• ps

• readlink

• lsns از package util-linux؛ اگر موجود نیست، مرحله مربوط به آن را skip کن.

۱۵.۲ قبل از اجرا: پیش‌بینی

قبل از commandها جواب این سوال‌ها را روی کاغذ بنویس:

1. آیا process داخل container در ps خود host دیده می‌شود؟
2. PID داخل container و PID همان process روی host باید یکی باشد؟
3. آیا همه namespaceهای container حتماً با host فرق دارند؟
4. آیا container یک kernel جدا دارد؟

جواب‌ها را بعد از آزمایش اصلاح کن.

۱۵.۳ یک container ساده بساز

```
docker run -d --name ds-dkr01 busybox:latest \
sh -c 'while true; do sleep 3600; done'
```

وضعیت را ببین:

```
docker ps --filter name=ds-dkr01
```

۱۵.۴ PID process اصلی را از دید host پیدا کن

```
HOST_PID=$(docker inspect -f '{{.State.Pid}}' ds-dkr01)
echo "$HOST_PID"
ps -o pid,ppid,user,comm,args -p "$HOST_PID"
```

نکته اصلی: process واقعاً در host process table وجود دارد. container یک VM مخفی با kernel جدا نیست.

۱۵.۵ داخل container چه می‌بینی؟

```
docker exec ds-dkr01 sh -c 'echo "inside: "; ps'
```

process اصلی داخل PID namespace خودش معمولاً PID 1 می‌بیند، در حالی که host برای همان PID process دیگری دارد.

این دو عدد تناقض ندارند. namespace view متفاوت ساخته است.

۱۵.۶ ID namespace ها را مقایسه کن

اول shell فعلی host:

```
for ns in mnt pid net uts ipc user cgroup; do
  printf '%-8s ' "$ns"
  readlink "/proc/$$/ns/$ns" 2>/dev/null || true
done
```

بعد process کاننتینر:

```
for ns in mnt pid net uts ipc user cgroup; do
  printf '%-8s ' "$ns"
  readlink "/proc/$HOST_PID/ns/$ns" 2>/dev/null || true
done
```

اگر lsns داری:

```
lsns -p "$HOST_PID"
```

۱۵.۷ چه چیزی باید مشاهده کنی؟

یک جدول برای محیط خودت پر کن:

برداشت	Same/Different	Container process ID	Host ID	Namespace
				pid
				mnt
				net
				uts
				ipc
				user
				cgroup

همه namespace ها لازم نیست متفاوت باشند. Docker configuration، rootless/usersns mode، host version و settings روی این موضوع اثر دارند. هدف آزمایش این نیست که یک output از قبل تعیین شده را حفظ کنی؛ هدف این است که configuration واقعی را مشاهده کنی.

۱۵.۸ kernel را مقایسه کن

روی host:

```
uname -a
```

داخل container:

```
docker exec ds-dkr01 uname -a
```

روی Linux process-isolated container معمولاً kernel identity نشان می‌دهد که container از kernel host استفاده می‌کند. بعضی field ها یا hostname ممکن است به دلیل namespace/configuration متفاوت دیده شوند؛ output را تحلیل کن، فقط مقایسه ظاهری نکن.

۱۵.۹ آزمایش اختیاری: یک مرز isolation را عمداً share کن

روی Linux:

```
docker run --rm --pid=host busybox:latest ps
```

قبل از اجرا پیش‌بینی کن: اگر PID namespace host را share کنیم، داخل container چه process‌هایی دیده می‌شوند؟ این آزمایش نشان می‌دهد binary isolation نیست؛ configuration می‌تواند یک boundary را باز کند.

۱۵.۱۰ پاک‌سازی

```
docker rm -f ds-dkr01
```

۱۵.۱۱ گزارش lab

در repo شخصی یا note خودت این پنج مورد را ثبت کن:

1. PID داخل container و PID host؛
2. namespace‌هایی که متفاوت بودند؛
3. namespace‌هایی که مشترک بودند؛
4. یک چیزی که برخلاف پیش‌بینی‌ات بود؛
5. توضیح یک پاراگرافی: «چرا این آزمایش نشان می‌دهد VM lightweight container نیست؟»

۱۶. پنج مدل ذهنی اشتباه

در این درس failure یعنی failure در تفکر، نه فقط error command.

۱۶.۱ «Container همان VM سبک است»

مشکل این مدل: guest kernel و hypervisor را نادیده می‌گیرد و باعث می‌شود در debugging دنبال machine boundary اشتباه بگردد.

اصلاح: container را process-based isolation ببین؛ VM را machine/kernel isolation.

۱۶.۲ «Container یعنی namespace»

مشکل: credential، mount، resource control و security policy حذف می‌شوند.

اصلاح: namespace فقط یکی از primitive‌هاست.

۱۶.۳ «Container کاملاً self-contained است»

مشکل: kernel، architecture، device، network و storage host را فراموش می‌کنی.

اصلاح: self-contained userspace تر است، نه کل execution universe.

۱۶.۴ «اگر داخل container root باشم یعنی root host هستم»

این جمله ممکن است در بعضی configuration ها خطرناک باشد، اما همیشه دقیق نیست. /User namespace

rootless، capability و security policy روی قدرت واقعی process اثر دارند.

اصلاح: username را با effective privileges اشتباه نگیر.

۱۶.۵ «Docker و container یک چیزند»

مشکل: استاندارد runtime، OCI، های دیگر و primitive های OS را نمی‌بینی.

اصلاح: Docker یک product/engine/ecosystem برای کار با containerهاست؛ container concept مستقل تر است.

۱۷. سوال‌های پیش‌بینی و چالش

بدون نگاه به جواب، برای هر مورد استدلال کن.

چالش ۱

Host یک Linux kernel دارد. داخل container یک Ubuntu userspace می‌بینی. آیا این Ubuntu kernel container خودش را دارد؟ چرا؟

چالش ۲

دو container از یک image ساخته شده‌اند. یکی `--network=host` دارد و یکی `network namespace` جدا. آیا می‌توان گفت isolation آن‌ها یکسان است؟

چالش ۳

یک image روی لپ‌تاپ `amd64` اجرا می‌شود ولی روی `server arm64` fail می‌شود. آیا این با ادعای portability تناقض دارد؟

چالش ۴

اگر cgroup memory limit برداشته شود ولی namespace ها همان باشند، آیا هنوز container داریم؟ چه چیزی تغییر کرده؟

چالش ۵

اگر Kubernetes از containerd استفاده کند و Docker Engine روی node نصب نباشد، آیا container image هایی که با Docker build شده اند دیگر قابل استفاده نیستند؟ برای جواب، image standard و runtime را از product Docker جدا کن.

چالش ۶

چرا process داخل container می تواند PID 1 ببیند در حالی که همان process روی host PID 42318 دارد؟

۱۸. پاسخ مصاحبه در سه عمق

۱۸.۱ سوال: Container چیست؟ - پاسخ ۳۰ ثانیه

کانتینر یک محیط اجرای ایزوله برای process است. در Linux معمولاً kernel جدا ندارد؛ runtime با primitive هایی مثل namespace، cgroup، mount و policy های امنیتی view و resource های process را کنترل می کند. از دید Docker، container یک instance قابل اجرای image به همراه runtime configuration است.

۱۸.۲ پاسخ حدود ۳ دقیقه

یک container را بهتر است lightweight VM نبینیم. VM یک guest OS با kernel خودش روی virtual hardware دارد، ولی container Linux application معمولاً process یا مجموعه process هایی روی kernel host است. Namespace ها view هایی مثل mount، PID و network را جدا می کنند؛ cgroup ها resource را کنترل و حساب می کنند؛ mount ها view root filesystem را می سازند؛ seccomp و capability، LSM ها روی privilege اثر دارند. Docker این primitive ها را با networking، API، lifecycle، image و storage workflow قابل استفاده می کند. Image و container هم یکی نیستند: image artifact است و container runtime instance آن به همراه configuration. Shared kernel باعث density و startup بهتر می شود، ولی portability و security boundary را هم به kernel host وابسته می کند.

۱۸.۳ پاسخ ۳۰ دقیقه‌ای: نقشه بحث

اگر interviewer اجازه deep dive داد، بحث را این ترتیب باز کن:

1. مسئله packaging و dependency drift؛
2. process-based isolation در برابر hardware virtualization؛
3. namespace vs cgroup vs mount vs security policy؛
4. shared kernel و syscall boundary؛
5. image vs runtime instance؛
6. portability limits: OS/kernel/architecture/device؛
7. Docker Engine و ecosystem؛
8. OCI standardization؛
9. containerd/runc boundary؛
10. security boundary و threat model؛
11. یک مثال واقعی از `/proc/$PID/ns` و PID mapping.

این ساختار مهم‌تر از حفظ کردن یک متن طولانی است.

۱۹. تمرین دونفره: دفاع و نقد (Attack / Defend)

با شریک مطالعات claim‌های زیر را یکی یکی بررسی کنید. یک نفر باید از claim دفاع کند و نفر دیگر آن را attack کند. بعد نقش‌ها عوض شود.

1. «Container یک VM سبک است.»
 2. «Docker بدون namespace معنی ندارد، پس container همان namespace است.»
 3. «اگر image ثابت باشد، application روی هر host دقیقاً یک رفتار دارد.»
 4. «Kubernetes جای Docker را گرفته است.»
 5. «Root داخل container یعنی root واقعی machine.»
 6. «چون container kernel مشترک دارد، isolation واقعی وجود ندارد.»
- هدف برنده شدن در بحث نیست. هدف پیدا کردن boundary هر گزاره است.

۲۰. سوال‌های بازگویی (Teach-back)

این سوال‌ها را شریک مطالعات باید از تو بپرسد. هنگام جواب دادن PDF بسته باشد.

1. اگر container یک kernel primitive واحد نیست، چه چیزهایی آن را می‌سازند؟
2. namespace و cgroup چه تفاوتی دارند؟

3. چرا container از process شروع می‌شود ولی VM از machine abstraction؟
4. shared kernel چه مزیت‌هایی دارد؟
5. shared kernel چه محدودیت‌هایی دارد؟
6. portability دقیقاً تا کجا می‌رود؟
7. image و container چه فرقی دارند؟
8. system container با Docker-style application container چه تفاوتی دارد؟
9. OCI چه مشکلی را حل می‌کند؟
10. Moby چیست و چرا برای contributor مهم است؟
11. containerd با runc یکی است؟
12. Kubernetes runtime است یا orchestrator؟
13. چرا security container را نباید فقط با کلمه «isolated» توصیف کرد؟
14. در lab چرا PID process روی host و داخل container فرق داشت؟
15. چه namespace‌هایی در محیط تو مشترک ماندند و چرا ممکن است این موضوع طبیعی باشد؟

۲۱. چک‌لیست تسلط

این درس را فقط وقتی Done علامت بزن که بیشتر موارد زیر واقعاً evidence داشته باشند.

فهم مفهومی

- ☐ . می‌توانم مشکل اصلی containerization را بدون slogan توضیح بدهم.
- ☐ . system container، application container، process و VM را از هم جدا می‌کنم.
- ☐ . می‌توانم توضیح بدهم چرا container یک primitive واحد kernel نیست.
- ☐ . namespace و cgroup را با نقش‌های متفاوت توضیح می‌دهم.
- ☐ . shared kernel را هم مزیت و هم محدودیت می‌بینم.
- ☐ . portability را بدون «هرجا اجرا می‌شود» توضیح می‌دهم.
- ☐ . image را با container اشتباه نمی‌گیرم.

نقشه اکوسیستم

- ☐ . Docker Engine را از Docker company/product جدا می‌کنم.

- ☐ . Moby را به عنوان upstream/open-source building blocks می‌شناسم.
- ☐ . OCI را به عنوان standardization layer توضیح می‌دهم.
- ☐ . containerd و runc را در دو سطح متفاوت قرار می‌دهم.
- ☐ . می‌دانم Compose runtime سطح پایین نیست.
- ☐ . می‌دانم Kubernetes orchestrator است و برای اجرای container به runtime نیاز دارد.

Lab

- ☐ . process اصلی container را در host process table پیدا کردم.
- ☐ . PID داخل و بیرون container را مقایسه کردم.
- ☐ . ID namespaces های host و container را ثبت کردم.
- ☐ . نتیجه `uname` را تحلیل کردم.
- ☐ . حداقل یک boundary را با configuration متفاوت آزمایش کردم یا دلیل skip را ثبت کردم.

ارتباط و مصاحبه

- ☐ . پاسخ ۳۰ ثانیه‌ای را بدون حفظ کردن جمله به جمله می‌دهم.
- ☐ . پاسخ ۳ دقیقه‌ای ساختار منطقی دارد.
- ☐ . می‌توانم ۳۰ دقیقه درباره موضوع deep dive کنم و مرز بحث را حفظ کنم.
- ☐ . partner من حداقل پنج boundary question پرسیده و جواب‌هایم ثبت شده‌اند.

Gate واقعی: اگر partner بپرسد «پس یک container دقیقاً کجای kernel ذخیره شده؟» نباید دنبال object به نام container بگردی. باید توضیح بدهی که state کانتینر بین namespace، process state، cgroupها، mountها و runtime metadata پخش شده است.

۲۲. تمرین خواندن منبع و Spec

برای این module لازم نیست spec را کامل بخوانی. فقط این بخش‌ها را مرور کن:

1. Docker Docs - صفحه «What is a container?» برای تعریف user-facing container و مقایسه با VM. [R1]
2. Docker Overview - بخش Architecture، Containers و Underlying technology. [R2]
3. Linux namespaces(7) - پاراگراف تعریف namespace و کاربرد آن برای container. [R4]
4. Linux kernel cgroup v2 docs - بخش «What is cgroup?». [R6]
5. OCI overview - دلیل وجود OCI و سه specification اصلی. [R9]
6. Linux OCI runtime config - فقط بخش namespace و cgroup را scan کن. [R11]
7. README پروژه‌های containerd، Moby، و runc - فقط mission و boundary هر پروژه. [R10][R15][R16]

هدف این reading این است که بتوانی حرف PDF را به source اصلی وصل کنی.

۲۳. سوال‌های باز که فعلاً باید نگه داری

این سوال‌ها عمده‌اً در درس‌های بعدی جواب کامل می‌گیرند:

- Docker دقیقاً چگونه namespace‌ها را ایجاد می‌کند؟
- `docker run` از CLI تا kernel چه مسیر code و API طی می‌کند؟
- containerd چرا shim دارد؟
- `runc` هنگام `create` چه syscalls می‌زند؟
- root filesystem کانتینر چگونه از image layer ساخته می‌شود؟
- network namespace چگونه به host وصل می‌شود؟
- PID 1 داخل container چه رفتار خاصی دارد؟
- rootless Docker چگونه بدون host root کار می‌کند؟

این سوال‌ها را حذف نکن. آن‌ها dependency graph یادگیری تو هستند.

۲۴. جمع‌بندی یک صفحه‌ای

اگر بخواهی کل درس را در چند خط نگه داری:

- کانتینر در مدل Docker یک محیط اجرای isolated برای process است، نه یک machine کامل.
- Linux container معمولاً kernel host را share می‌کند.
- container از کنار هم قرار گرفتن primitive‌های مختلف ساخته می‌شود؛ namespace فقط یکی از آن‌هاست.

- namespace view را جدا می‌کند؛ cgroup resource organization/control را انجام می‌دهد؛ mountها
- filesystem view را می‌سازند؛ credentials و security policy privilege را محدود می‌کنند.
- shared kernel باعث density و startup بهتر می‌شود ولی compatibility و security را به host kernel گره می‌زند.
- portability واقعی است ولی مطلق نیست؛ OS/kernel/architecture/device/network/storage هنوز قرارداد اجرا هستند.
- image artifact است؛ container runtime instance است.
- Docker Engine، Moby، OCI، containerd و runc لایه‌های مختلف‌اند.
- Kubernetes orchestrator است، نه جایگزین مفهوم container.
- بهترین اثبات فهم این درس این است که process کانتینری را در host پیدا کنی و namespace‌هایش را مشاهده کنی.

۲۵. قدم بعدی

بعد از پاس‌کردن Mastery Checklist، مسیر مستقیم به:

DKR.02 - Editions، Platforms و Distribution، نصب

می‌رود.

در DKR.02 محیط واقعی مطالعه را استاندارد می‌کنی، version‌ها را ثبت می‌کنی، فرق Docker و Docker Engine را Desktop را عملی می‌فهمی و remote context/daemon boundary را می‌بینی. بعد از آن DKR.03 معماری Engine را از CLI تا runtime stack باز می‌کند.

منابع

این سند از source‌های اولیه و رسمی زیر ساخته شده است. تاریخ بررسی همه منابع: 2026-08-11.

?R1 - Docker Docs: What is a container

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

تعریف user-facing کانتینر، process isolation و مقایسه container با VM.

?R2 - Docker Docs: What is Docker

<https://docs.docker.com/get-started/docker-overview/>

underlying Linux namespace technology و Docker platform، Engine architecture، container object

?R3 – Docker Docs: What is an image

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>

تعریف image به عنوان package شامل files/binaries/libraries/configuration و تفاوت آن با runtime container.

R4 – Linux man-pages: namespaces(7)

<https://man7.org/linux/man-pages/man7/namespaces.7.html>

تعریف namespace و نقش آن در ساخت view جدا از global resource؛ اشاره صریح به container implementation.

R5 – Kubernetes Docs: Containers

<https://kubernetes.io/docs/concepts/containers/>

Container به عنوان packaging application و dependency، repeatability و توضیح runtime در Kubernetes.

R6 – Linux Kernel Documentation: Control Group v2

<https://docs.kernel.org/admin-guide/cgroup-v2.html>

تعریف cgroup به عنوان process hierarchy و resource distribution/control.

R7 – Linux Containers / Incus: Containers and VMs

https://linuxcontainers.org/incus/docs/main/explanation/containers_and_vms/

تفکیک system container، application container و VM.

?R8 – Docker: What is a Container

<https://www.docker.com/resources/what-container/>

تاریخ launch Docker در ۲۰۱۳، استفاده از cgroups/namespaces و نقش Docker در developer/operator workflow.

R9 – Open Container Initiative: Overview

<https://opencontainers.org/about/overview/>

شکل‌گیری OCI در ۲۰۱۵ و سه specification اصلی Image، Runtime و Distribution.

R10 – Moby Project README

<https://github.com/moby/moby>

تعریف Moby به عنوان modular open-source project و رابطه upstream آن با Docker product.

R11 – OCI Runtime Specification: Linux configuration

<https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>

Runtime configuration مربوط به namespace، Linux، ها و cgroupها.

R12 – Microsoft Learn: Containers vs Virtual Machines

<https://learn.microsoft.com/en-us/virtualization/windowscontainers/about/containers-vs-vm>

مقایسه architecture کانتینر و VM از دید kernel و isolation.

R13 – Microsoft Learn: Windows Container Isolation Modes

<https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/hyperv-container>

Process isolation در برابر Hyper-V isolation و shared-kernel behavior در Windows.

R14 – Docker Docs: Docker Engine

<https://docs.docker.com/engine/>

تعریف Engine به عنوان client-server containerization technology شامل API، daemon، و CLI.

R15 – runc README

<https://github.com/opencontainers/runc>

runc به عنوان low-level Linux OCI runtime tool.

R16 – containerd README

<https://github.com/containerd/containerd>

containerd به عنوان daemon/runtime سطح بالاتر برای image transfer/storage، lifecycle، و execution /supervision.

R17 - Kubernetes Docs: Container Runtimes

<https://kubernetes.io/docs/setup/production-environment/container-runtimes/>

نیاز Kubernetes به CRI-compatible runtime و حذف dockershim از Kubernetes 1.24.

R18 - Docker Engine 29 Release Notes

<https://docs.docker.com/engine/release-notes/29/>

Baseline فعلی پروژه: Docker Engine 29.7.2 در 2026-08-05؛ همچنین نشانه‌های version-sensitive runtime topology در شاخه 29.

R19 - Microsoft Learn: Secure Windows Containers

<https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/container-security>

تفاوت shared-kernel isolation و hypervisor security boundary برای hostile multi-tenant workloads.

Changelog

2026-08-11 - 1.0.0

- اولین نسخه canonical درس DKR.01.
- ساخته شده بر اساس Docker Mastery Syllabus v1.1.0.
- شامل Scope Contract، مدل ذهنی، تاریخچه، lab، ecosystem map واقعی، interview ladder، failure pack، teach-back و Mastery Checklist.
- متن فارسی با اولویت سادگی و استفاده کنترل شده از اصطلاحات انگلیسی نوشته شده است.